VesperaRoom — Documentación del backend (modelo 3FN completo)

Oct 27, 2026 · @Felipe

Visión general

El backend es una API REST en Spring Boot 4.1.1 con Java 21 que accede a MySQL solo con JDBC, sin ORM. El SQL está escrito a mano en el repositorio y no hay ninguna anotación de mapeo como `@Entity`. Implementa las 19 tablas del modelo normalizado a 3FN (catálogos, moteles, habitaciones, tarifas, reservas, pagos y reseñas; ver la sección «Módulos del modelo 3FN»). El primer módulo fue **Usuario**: registro, inicio de sesión con JWT y administración de cuentas.

Cada petición atraviesa las mismas capas, siempre en el mismo orden:

1. **Seguridad**: el filtro JWT identifica al usuario y `SecurityConfig` decide si tiene permiso para el endpoint.
2. **Controller**: recibe el JSON, llama al servicio y traduce el resultado a una respuesta HTTP.
3. **Service**: valida los datos con `LanzadorExcepciones`, aplica las reglas de negocio y devuelve un código `int`.
4. **Repository**: abre una conexión JDBC, ejecuta el SQL con `PreparedStatement` y convierte cada fila del `ResultSet` en un objeto `Usuario`.
5. **MySQL**: la tabla `usuario` y sus restricciones (`UNIQUE`, `CHECK`) son la última línea de defensa.



recorrido de una petición · 5 capas y 3 clases de apoyo

La respuesta recorre el camino inverso: el código `int` del servicio sube al controller, que lo convierte en HTTP con `RespuestaUsuario`.

Paquete	Responsabilidad	Clases
raíz	Arranque de la aplicación	`VesperaroomApplication`
`configuration`	Conexión JDBC, beans de arranque y documentación Swagger	`ConexionBD`, `LoadDataBase`, `OpenApiConfig`
`controller`	Endpoints REST	`AuthController`, `UsuarioController`
`dto`	Objetos que viajan en el JSON	`UsuarioDTO`, `LoginDTO`, `AuthResponseDTO`
`entity`	Representación en Java de una fila de la tabla	`Usuario`
`exception`	Validaciones y excepciones propias	`LanzadorExcepciones` + 12 excepciones

Paquete	Responsabilidad	Clases
`repository`	SQL y acceso JDBC	`UsuarioRepository`
`security`	JWT, filtro y reglas de acceso	`JwtUtil`, `JwtAuthenticationFilter`, `SecurityConfig`, `UserDetailsServiceImpl`
`service`	Lógica de negocio	`CRUDOperation`, `UsuarioService`
`util`	Traducción de códigos a HTTP	`RespuestaUsuario`

Cada paquete tiene su `package-info.java`, que lo describe en el Javadoc.

Configuración JDBC

La conexión se configura en una clase propia, `ConexionBD`, y todo el SQL se ejecuta con JDBC puro en `UsuarioRepository`: `Connection`, `PreparedStatement` y `ResultSet` escritos a mano. No hay ORM ni `JdbcTemplate`; la librería `spring-jdbc` ni siquiera está en el proyecto.

1. Dependencias (`pom.xml`)

Dependencia	Qué aporta
`HikariCP`	El pool de conexiones. Implementa la interfaz estándar `javax.sql.DataSource`
`mysql-connector-j` (scope `runtime`)	El driver JDBC de MySQL (`com.mysql.cj.jdbc.Driver`)

No están `spring-boot-starter-data-jpa` (ORM) ni `spring-boot-starter-jdbc` (`JdbcTemplate`). También se retiró `spring-boot-devtools`, porque su autoconfiguración exige el módulo `spring-boot-jdbc` cuando existe un `DataSource`.

2. Datos de conexión (`src/main/resources/application.properties`)

```
spring.datasource.url=jdbc:mysql://${DB_HOST:localhost}:${DB_PORT:3306}/motel
now
spring.datasource.username=${DB_USER:usuario}
spring.datasource.password=${DB_PASSWORD}
spring.sql.init.mode=never
```

`${VAR:valor}` lee una variable de entorno y usa el valor por defecto si no existe. La contraseña no tiene valor por defecto, para no dejarla en el repositorio. Spring no crea tablas: el esquema sale de los scripts de `/database`.

3. La conexión: `configuration/ConexionBD.java`

Esta clase construye el pool de conexiones a mano en su método `dataSource()`, marcado con `@Bean`:

1. Lee la url, el usuario y la contraseña de las propiedades con `@Value`.
2. Crea un `HikariConfig` y le asigna el driver (`setDriverClassName`), la url (`setJdbcUrl`), las credenciales, el máximo de 10 conexiones (`setMaximumPoolSize`) y una espera máxima de 5 segundos por conexión (`setConnectionTimeout`).
3. `setInitializationFailTimeout(-1)` hace que el pool no se conecte al arrancar, sino con la primera consulta. Así la aplicación inicia aunque MySQL esté apagado.
4. Retorna un `HikariDataSource`. Spring lo inyecta en el constructor de `UsuarioRepository` y lo cierra al apagar la aplicación (`destroyMethod = "close"`).

El pool mantiene conexiones abiertas y las presta. `dataSource.getConnection()` toma una conexión prestada y `con.close()` la devuelve al pool en lugar de cerrarla. Por eso abrir y cerrar una conexión en cada método no cuesta un nuevo login a MySQL.

4. Las consultas: `repository/UsuarioRepository.java`

Cada método sigue el ciclo completo de JDBC. Ejemplo real, `findByCorreo`:

```
public Optional<Usuario> findByCorreo(String correo) throws SQLException {
    try (Connection con = dataSource.getConnection();
// 1. pedir conexión al pool
         PreparedStatement ps = con.prepareStatement(SELECT_POR_CORREO)) {
// 2. preparar el SQL con ?
        ps.setString(1, correo);
// 3. asignar el parámetro
        try (ResultSet rs = ps.executeQuery()) {
// 4. ejecutar
            if (rs.next()) {
// 5. leer la fila
                return Optional.of(mapear(rs));
            }
            return Optional.empty();
        }
    }
// 6. cerrar todo
}
```

1. `getConnection()` pide una conexión al pool de `ConexionBD`.
2. `prepareStatement` envía a MySQL el SQL con marcadores `?`. El SQL está en constantes al inicio de la clase (`SELECT_BASE`, `SELECT_POR_CORREO`, `INSERT`, `UPDATE`, `DELETE`...).
3. `setString`, `setLong` y `setTimestamp` asignan cada `?` por posición (1, 2, 3...). Los valores viajan aparte del SQL, así que no hay inyección SQL.
4. `executeQuery()` ejecuta un `SELECT` y devuelve un `ResultSet`; `executeUpdate()` ejecuta `INSERT`, `UPDATE` o `DELETE` y devuelve las filas afectadas.
5. `rs.next()` avanza fila por fila. El método privado `mapear(rs)` lee cada columna por nombre (`rs.getString("correo")`, `rs.getTimestamp("fecha_registro")`) y arma el `Usuario`.
6. El `try-with-resources` cierra `ResultSet`, `PreparedStatement` y `Connection` en orden inverso, aunque ocurra un error. Cerrar la conexión la devuelve al pool.

Operación	Llamadas JDBC	Métodos
`SELECT` de varias filas	`executeQuery()` + `while (rs.next())`	`findAll`, `findByRol`
`SELECT` de una fila	`executeQuery()` + `if (rs.next())` → `Optional`	`findById`, `findByCorreo`
`SELECT COUNT(*)`	`executeQuery()` + `rs.getLong(1)`	`contar` → `existsBy...`, `count`
`INSERT`	`prepareStatement(sql, RETURN_GENERATED_KEYS)` + `executeUpdate()` + `getGeneratedKeys()` para leer el `id_usuario` del `AUTO_INCREMENT`	`insertar`
`UPDATE` / `DELETE`	`executeUpdate()` → filas afectadas	`actualizar`, `actualizarEstado`, `eliminar`

5. Manejo de errores

Los métodos del repositorio declaran `throws SQLException`: no ocultan el error, lo pasan al servicio.

- `SQLIntegrityConstraintViolationException` con código MySQL 1062 (llave duplicada): el servicio lee el nombre de la restricción y devuelve 8 (correo) o 9 (teléfono).

- La misma excepción al eliminar (código 1451, la fila tiene registros que dependen de ella): devuelve 2.
- Cualquier otra `SQLException` en una escritura, como MySQL caído: devuelve 11.
- En las lecturas (`getAll` , `getById` …), que no pueden devolver un código, se envuelve en `AccesoDatosException` y Spring responde 500.
- En el login, `UserDetailsServiceImpl` la envuelve en `InternalAuthenticationServiceException` y la respuesta es 500 "Error interno al autenticar".

6. Transacciones

MySQL trabaja en modo *autocommit*: cada sentencia se confirma sola. Hoy cada operación es una sola escritura, así que no hace falta más. Cuando una operación escriba en varias tablas (por ejemplo, reserva + pago), se hará con la misma conexión:

```
try (Connection con = dataSource.getConnection()) {
    con.setAutoCommit(false);
    try {
        // varios PreparedStatement sobre la misma con
        con.commit();
    } catch (SQLException e) {
        con.rollback();
        throw e;
    }
}
```

Módulos del modelo 3FN

Las 18 tablas nuevas siguen exactamente el patrón de Usuario: entidad POJO, DTO, repositorio JDBC puro (`throws SQLException`), servicio con códigos `int` documentados en su Javadoc, clase `Respuesta<Entidad>` con el `switch` y controlador. Las tablas intermedias no tienen controlador propio: `tipo_habitacion_caracteristica` la administra `TipoHabitacionRepository` y `reserva_servicio` la administra `ReservaRepository` .

Entidad	Ruta base	Quién escribe	Reglas principales
Departamento, Ciudad, Caracteristica, Modalidad	`/departamento`, `/ciudad`, `/caracteristica`, `/modalidad`	ADMINISTRADOR	Catálogos con nombre único (la ciudad, único dentro de su departamento). Consulta pública
PerfilPropietario	`/perfilpropietario`	El propietario (`/mi-perfil`) o el administrador	PK = `id_usuario`; NIT y cuenta bancaria únicos; solo para usuarios con rol PROPIETARIO; el administrador lo verifica con `/verificar`
Motel	`/motel`	Propietario dueño o administrador	Requiere perfil de propietario y ciudad existentes; `/getall` público solo muestra moteles ACTIVO
PoliticaReserva	`/politica`	Propietario dueño	PK = `id_motel`; `aceptaReembolso` se calcula (`porcentaje > 0`), no se guarda
TipoHabitacion	`/tipohabitacion`	Propietario dueño	Nombre único en el motel; asigna y quita características (N:M)
Habitacion	`/habitacion`	Propietario dueño	El motel se toma del tipo, nunca del cliente; número único en el motel; `/disponibles` público

Entidad	Ruta base	Quién escribe	Reglas principales
Foto	`/foto`	Propietario dueño	Pertenece a un motel **o** a una habitación; posición (`orden`) única por dueño
Tarifa	`/tarifa`	Propietario dueño	Precio > 0; vigencia válida; única por tipo, modalidad, tipo de día y fecha
ServicioAdicional	`/servicio`	Propietario dueño	Nombre único en el motel; se desactiva en lugar de borrarse si ya se vendió
Reserva	`/reserva`	Cualquier usuario autenticado (a su nombre)	Total, hora de fin y código los calcula el servidor; se guarda en una transacción; solo cambia de estado
Pago	`/pago`	Cliente de la reserva o propietario	La suma de pagos no supera el total; el propietario aprueba, rechaza o reembolsa
Resena	`/resena`	Cliente dueño de la reserva	Solo reservas FINALIZADA, una por reserva; promedio público con `AVG`
RespuestaPropietario	`/respuesta`	Propietario del motel reseñado	PK = `id_resena` (entidad débil 1:1)

Todos los controladores usan las rutas de TravelX: `/createjson` , `/getall` , `/getbyid/{id}` , `/updatejson?id=` , `/deletebyid/{id}` , `/count` y `/exists/{id}` , más las consultas propias de cada entidad (`/getbymotel/{id}` , `/mis-reservas` , `/updateestado` …).

Autorización por propiedad

`SecurityConfig` solo sabe de roles: deja pasar a cualquier PROPIETARIO a `/motel/**`, pero no sabe de quién es cada motel. Esa segunda verificación la hace `AutorizacionService`, que cada controlador llama antes de modificar algo:

1. El controlador recibe el usuario autenticado con `@AuthenticationPrincipal Usuario actual`.
2. Llama, por ejemplo, a `autorizacionService.puedeGestionarHabitacion(actual, id)`.
3. `AutorizacionRepository` recorre las llaves foráneas con `JOIN` hasta el motel y devuelve su `id_propietario`.
4. Si no coincide con el usuario, el controlador responde 403 con `RespuestaGeneral.prohibido()`.

El administrador siempre pasa. Si el recurso no existe, también pasa, para que el servicio responda 404 y no un 403 engañoso. En la creación de moteles, perfiles y reservas, el controlador sobrescribe el dueño con el id del token, así un usuario no puede crear registros a nombre de otro.

Clases de apoyo nuevas (`util`)

Clase	Para qué sirve
`RespuestaGeneral`	Respuestas que se repetían en todos los controladores: listas (202/204), búsqueda por id (202/404), eliminación (202/404/409), cambio de estado, conteo, existencia y acceso denegado (403)
`Respuesta<Entidad>`	Un `switch` por entidad, igual que `RespuestaUsuario`, que traduce los códigos del servicio a mensaje y código HTTP
`Normalizador`	`texto()` quita espacios sobrantes; `textoOpcional()` además convierte el texto vacío en `null` para guardar `NULL` en la base
`CodigoMySQL`	Constantes de los códigos de error de MySQL (1062 llave duplicada, 1451 tiene dependientes, 1452 referencia inexistente) y métodos para reconocerlos

Creación de una reserva paso a paso

`ReservaService.crear` es el método con más reglas del proyecto. El cliente solo envía habitación, tarifa, fecha de inicio, cantidad de personas y servicios; todo lo demás lo calcula el servidor:

1. Valida personas (1 a 20) y que la fecha de inicio sea futura.
2. Busca la habitación y su motel; ambos deben estar activos.
3. Busca la tarifa: debe ser del mismo tipo que la habitación y estar vigente en esa fecha.
4. Si el motel tiene política, exige la anticipación mínima en horas.
5. Compara las personas con la capacidad del tipo de habitación.
6. Valida cada servicio (existe, es del mismo motel, está activo, cantidad 1 a 20, no repetido) y toma su precio vigente como `precio_unitario`.
7. Calcula la hora de fin: inicio + `duracion_horas` de la modalidad de la tarifa.
8. Comprueba que no haya otra reserva vigente que se cruce con ese horario.
9. Calcula el total: precio de la tarifa + recargo + Σ (precio × cantidad) de los servicios. El recargo es `(personas − capacidad_max_sin_recargo) × tarifa_persona_adicional` cuando hay más personas de las permitidas sin recargo.
10. Genera un código como `VR-7KQ2M9XA` y guarda todo con `ReservaRepository.insertarConServicios`.

El paso 10 es una transacción JDBC escrita a mano. Es el primer uso real del patrón de la sección «6. Transacciones»:

```
try (Connection con = dataSource.getConnection()) {
    con.setAutoCommit(false);                // inicia la transacción
    try {
        // 1. SELECT id_habitacion FROM habitacion WHERE id_habitacion = ?
FOR UPDATE
        // 2. si hay cruce de horario: con.rollback(); return null;
        // 3. INSERT INTO reserva ... (RETURN_GENERATED_KEYS)
        // 4. INSERT INTO reserva_servicio ... (addBatch / executeBatch)
        con.commit();                        // todo o nada
        return idReserva;
    } catch (SQLException e) {
        con.rollback();                      // deshace lo que alcanzó a
hacer
        throw e;
    } finally {
        con.setAutoCommit(true);             // la conexión vuelve limpia al
pool
    }
}
```

`SELECT ... FOR UPDATE` bloquea la fila de la habitación hasta el `commit`. Si dos clientes reservan la misma habitación al mismo tiempo, el segundo espera a que el primero termine y, al repetir la comprobación del paso 8, ve el cruce y no inserta. Sin ese bloqueo, ambos podrían pasar la comprobación antes de que cualquiera insertara.

La reserva no se edita: solo cambia de estado con `cambiarEstado`, que respeta este ciclo de vida (los estados finales no cambian más). Por eso `ReservaService` y `PagoService` no implementan `CRUDOperation`: son registros históricos.

Estado actual	Puede pasar a
PENDIENTE	CONFIRMADA, CANCELADA
CONFIRMADA	EN_CURSO, CANCELADA, NO_SHOW
EN_CURSO	FINALIZADA
FINALIZADA, CANCELADA, NO_SHOW	(ninguno)

El pago sigue el mismo esquema: PENDIENTE → APROBADO o RECHAZADO, y APROBADO → REEMBOLSADO.

Cancelación, reembolso y estados automáticos

Cancelar ya no es un simple cambio de estado: tiene su propio método, `ReservaService.cancelar(id, porCliente)`, que calcula cuánto se le devuelve al cliente y lo deja guardado en la reserva. `cambiarEstado` ya no acepta CANCELADA; si el propietario manda CANCELADA por `/reserva/updateestado`, el controlador lo redirige a `cancelar`.

Quién recibe cuánto

Quién cancela	Ruta	Cuándo puede	Reembolso
Cliente	`PUT /reserva/cancelar/{id}`	Reserva PENDIENTE o CONFIRMADA y antes de su hora de inicio	`porcentajeReembolso` de la política del motel (0 % si el motel no tiene política)
Propietario o administrador	`PUT /reserva/updateestado?estado=CANCELADA`	Reserva PENDIENTE o CONFIRMADA, a cualquier hora	100 %: el cliente no tuvo la culpa

Quién cancela	Ruta	Cuándo puede	Reembolso
Tarea automática	(sin ruta)	Reserva PENDIENTE cuya hora de inicio llegó sin confirmar	100 %

Cálculo

1. Se suman solo los pagos **APROBADOS** con `PagoRepository.sumarMontoAprobado` (los pendientes nunca se cobraron).
2. `monto = pagado × porcentaje / 100`, redondeado a 2 decimales con `RoundingMode.HALF_UP`. Ejemplo con el motel 1 (80 %): 190.000 pagados → 152.000 de reembolso.
3. Si el monto es mayor que cero, la fecha límite es hoy + `diasHabilesDevolucion` días hábiles, saltando sábados y domingos con `DayOfWeek` (no descuenta festivos). Sin política se usan 5 días.
4. `ReservaRepository.cancelar` hace en **una transacción**: `UPDATE reserva SET estado = 'CANCELADA', fecha_cancelacion, monto_reembolso, fecha_limite_reembolso … WHERE estado IN ('PENDIENTE', 'CONFIRMADA')` y `UPDATE pago SET estado = 'RECHAZADO'` para los pagos pendientes. Si el primer UPDATE no afecta filas (otra petición cambió la reserva entre la lectura y la escritura), hace `rollback`.
5. El controlador vuelve a leer la reserva y responde, por ejemplo, «Reserva cancelada. Se devolverán $152000.00 a más tardar el 2026-11-06».

Los pagos aprobados **no** cambian a REEMBOLSADO: ese dinero sí entró, y lo que sale queda en `reserva.monto_reembolso`. Por eso la consulta de ingresos del script 03 ahora calcula **ingresos netos = pagos aprobados − reembolsos**, y hay una consulta nueva de reembolsos pendientes de devolver.

Códigos de `cancelar`: 0 éxito (202), 1 no encontrada (404), 2 no está PENDIENTE ni CONFIRMADA (409), 3 el cliente intenta cancelar una reserva que ya comenzó (409), 4 error inesperado (500). La regla 3 usa la validación nueva `LanzadorExcepciones.verificarAntesDelInicio`.

Cambio automático de estados

La clase `scheduler/ActualizadorEstadosReserva` tiene un método con `@Scheduled(fixedDelayString = "${app.reservas.intervalo-ms:60000}")`: Spring lo ejecuta en segundo plano cada 60 segundos (configurable en `application.properties`) gracias a `@EnableScheduling` en `VesperaroomApplication`. Llama a `ReservaService.actualizarEstadosAutomaticos(ahora)`:

Regla	Cómo se hace
PENDIENTE cuya hora de inicio ya llegó → CANCELADA con reembolso del 100 %	`findPendientesVencidas` trae los ids y cada uno pasa por `cancelar(id, false)`, para reutilizar el cálculo del reembolso
CONFIRMADA cuyo cliente no llegó dentro del tiempo de gracia → NO_SHOW, sin reembolso	Un solo `UPDATE reserva r JOIN habitacion h … LEFT JOIN politica_reserva p … WHERE DATE_ADD(r.fecha_hora_inicio, INTERVAL COALESCE(p.tiempo_gracia_no_show_min, 0) MINUTE) <= ?`
EN_CURSO cuya hora de fin pasó → FINALIZADA (y ya se puede reseñar)	`UPDATE reserva SET estado = 'FINALIZADA' WHERE estado = 'EN_CURSO' AND fecha_hora_fin <= ?`

El paso de CONFIRMADA a EN_CURSO sigue siendo **manual**: solo el propietario sabe cuándo llega el cliente. La hora se pasa como parámetro (`ahora`) en vez de usar `NOW()` de MySQL, para que Java y la base usen la misma hora y para poder probarlo con una fecha fija. Si la base falla, la tarea registra el error con SLF4J y lo intenta en la siguiente vuelta.

Cambios en la base: `reserva` tiene tres columnas nuevas y opcionales, `fecha_cancelacion`, `monto_reembolso` y `fecha_limite_reembolso`, con el CHECK `ck_reserva_reembolso` (el reembolso va de 0 al total). Dependen solo de `id_reserva`, así que el modelo sigue en 3FN. Para una base ya creada se aplica `migraciones/V3__reembolso_reserva.sql`; `R3` la revierte.

Funciones nuevas usadas aquí (no estaban en TravelX)

Función	Para qué
`@EnableScheduling`, `@Scheduled(fixedDelayString, initialDelayString)`	Ejecutar un método cada cierto tiempo; `fixedDelay` espera a que termine la vuelta anterior antes de contar el intervalo, así dos revisiones nunca se cruzan
`BigDecimal.divide(divisor, 2, RoundingMode.HALF_UP)`	Dividir dinero con 2 decimales y redondeo comercial; sin la escala, `divide` lanza error con resultados periódicos
`DayOfWeek`, `LocalDate.plusDays`, `getDayOfWeek()`	Contar días hábiles

Función	Para qué
`UPDATE … JOIN` y `DATE_ADD(fecha, INTERVAL n MINUTE)` (MySQL)	Actualizar reservas leyendo la política del motel en la misma sentencia
`Logger` / `LoggerFactory` (SLF4J, incluido en Spring Boot)	Escribir en la consola de la aplicación desde la tarea, que no tiene una petición HTTP a la que responder

Métodos y funciones que no estaban en TravelX

TravelX usaba JPA, así que todo lo de JDBC es nuevo frente a ese proyecto. Esta tabla reúne lo que conviene poder explicar en la sustentación.

Método o función	Dónde se usa	Qué hace y por qué
`rs.getObject("col", Long.class)`	`mapear` de los repositorios, en columnas que admiten `NULL`	Devuelve `null` si la columna es `NULL`. `rs.getLong()` devolvería 0, que es un valor falso
`ps.setObject(i, valor, Types.BIGINT)`	`insertar` y `actualizar` con columnas opcionales	Escribe `NULL` cuando el valor es `null`. `ps.setLong()` no acepta `null`
`rs.getDate().toLocalDate()` y `ps.setDate(i, Date.valueOf(fecha))`	Tarifas (`vigente_desde`)	Convierten entre `java.sql.Date` y `LocalDate`, igual que `Timestamp` con `LocalDateTime` en Usuario
`rs.wasNull()`	`ResenaRepository.promedioByMotel`	Indica si el último valor leído era `NULL`; `AVG` devuelve `NULL` cuando el motel no tiene reseñas
`con.setAutoCommit(false)`, `commit()`, `rollback()`	`ReservaRepository.insertarConServicios`	Agrupan varias sentencias en una transacción: se guardan todas o ninguna
`SELECT ... FOR UPDATE`	Misma transacción	Bloquea la fila hasta el `commit` para evitar dos reservas simultáneas de la misma habitación

Método o función	Dónde se usa	Qué hace y por qué
`ps.addBatch()` y `ps.executeBatch()`	Detalle de servicios de la reserva	Envía varios `INSERT` de una vez reutilizando el mismo `PreparedStatement`
`e.getErrorCode()` (1062, 1451, 1452)	`CodigoMySQL` y los servicios	Distingue llave duplicada, registro con dependientes y referencia inexistente cuando la base rechaza una operación
`NOT EXISTS`, `COALESCE`, `AVG`, `JOIN`	`HabitacionRepository.findDisponibles`, `AutorizacionRepository`, `PagoRepository`, `ResenaRepository`	Consultas que reconstruyen por `JOIN` lo que la normalización separó
`BigDecimal.add`, `multiply`, `compareTo`, `signum`	`ReservaService`, `LanzadorExcepciones`	Aritmética exacta de dinero; `double` acumula errores de redondeo. Se compara con `compareTo` porque `equals` distingue 90000 de 90000.00
`LocalDateTime.plusHours`, `isAfter`, `isBefore`	Hora de fin, fecha futura, anticipación	Operaciones de fecha y hora de `java.time`
`SecureRandom.nextInt`	`ReservaService.generarCodigo`	Genera el código de reserva con un aleatorio no predecible; `Random` sí se puede adivinar
`HashSet.add`	`ReservaService.prepararServicios`	Devuelve `false` si el servicio ya estaba en la lista, así se detectan servicios repetidos
`MatchingStrategies.STRICT`	Bean de `ModelMapper` en `VesperaroomApplication`	Solo copia atributos con el mismo nombre exacto. Con tantos campos `idMotel`, `idTipo`, `idHabitacion`, la estrategia por defecto podía confundirlos

Método o función	Dónde se usa	Qué hace y por qué
`@DateTimeFormat(iso = ISO.DATE_TIME)`	`HabitacionController.obtenerDisponibles`	Convierte el parámetro `2026-11-20T21:00:00` de la URL en `LocalDateTime`
`getCampo()`, `getEntidad()`, `getDetalle()` en excepciones	`TextoInvalidoException`, `ReferenciaNoEncontradaException`, `RegistroDuplicadoException`	Guardan qué campo o referencia falló, para que un mismo tipo de excepción pueda devolver códigos distintos (por ejemplo, ciudad inexistente frente a propietario sin perfil)
`@MockitoSettings(strictness = Strictness.LENIENT)`	`ReservaServiceTest`	Permite preparar un escenario completo en `@BeforeEach` aunque algunas pruebas terminen antes de usar todos los datos simulados

Pruebas

Hay 52 pruebas unitarias con JUnit 5 y Mockito, todas pasan: 24 de `UsuarioServiceTest`, 11 de `LanzadorExcepcionesTest` (incluidas las validaciones nuevas), 13 de `ReservaServiceTest` (cálculo del total y la hora de fin, cada código de error, horario ocupado dentro de la transacción, transiciones de estado, reembolso según quién cancela, cancelaciones no permitidas y la tarea automática) y 4 de `AutorizacionServiceTest`. Los demás servicios nuevos siguen el mismo patrón, pero no tienen pruebas unitarias propias. La seguridad (401/403 por rol) y las validaciones previas a la base se probaron con la aplicación corriendo; las operaciones sobre las tablas nuevas quedan por probar contra MySQL después de ejecutar los scripts.

Explicación de cada clase

Las clases aparecen en el orden en que interviene una petición, desde los datos hasta la seguridad. La ruta base de todas es `src/main/java/co/edu/unbosque/vesperaroom/`.

Aplicación

`VesperaroomApplication`: punto de entrada. `@SpringBootApplication` activa el escaneo de componentes, que encuentra `ConexionBD`, los repositorios, los servicios y los controladores. También declara el bean `ModelMapper`, que copia campos con el mismo nombre entre `Usuario` y `UsuarioDTO`.

Entidad (`entity`)

`Usuario`: un objeto Java simple que representa una fila de la tabla `usuario`, sin anotaciones de base de datos. El mapeo columna → atributo lo hace a mano el `RowMapper` del repositorio.

- Atributos: `idUsuario`, `nombre`, `apellido`, `correo`, `telefono`, `contrasena` (hash BCrypt), `rol`, `estado`, `fechaRegistro`.
- Enums anidados: `Rol` (`ADMINISTRADOR`, `CLIENTE`, `PROPIETARIO`) y `Estado` (`ACTIVO`, `INACTIVO`). En la BD se guardan como texto con `name()` y se leen con `valueOf()`.
- Implementa `UserDetails` para que Spring Security la use directamente: `getUsername()` devuelve el correo, `getPassword()` el hash, `getAuthorities()` devuelve `ROLE_<rol>` e `isEnabled()` solo es verdadero si el estado es `ACTIVO`.
- `toString()` excluye la contraseña para que no aparezca en los logs.

DTOs (`dto`)

Son los objetos que entran y salen en el JSON. Separarlos de la entidad evita exponer columnas internas.

Clase	Uso	Detalle
`UsuarioDTO`	Crear, registrar, actualizar y consultar usuarios	`idUsuario` y `fechaRegistro` son de solo lectura (`READ_ONLY`): el cliente no puede fijarlos. `contrasena` es de solo escritura (`WRITE_ONLY`): se recibe pero nunca se devuelve
`LoginDTO`	Cuerpo de `/auth/login`	Solo `correo` y `contrasena`; su `toString()` no muestra la contraseña
`AuthResponseDTO`	Respuesta de un login exitoso	`token`, `idUsuario`, `nombre`, `correo` y `rol` para el frontend

Repositorio (`repository`)

`UsuarioRepository`: la única clase que ejecuta SQL, con JDBC puro (ciclo completo en la sección de configuración JDBC). Recibe el `DataSource` de `ConexionBD` por constructor.

- Constantes con el SQL: `SELECT_BASE` (columnas comunes), `SELECT_TODOS`, `SELECT_POR_ROL`, `SELECT_POR_ID`, `SELECT_POR_CORREO`, `INSERT`, `UPDATE`, `UPDATE_ESTADO` y `DELETE`.
- Cada método abre su `Connection`, prepara el `PreparedStatement`, asigna los `?`, ejecuta y cierra todo con try-with-resources.
- `mapear(rs)`: método privado que lee cada columna del `ResultSet` por nombre (`rs.getString("correo")`, `Rol.valueOf(rs.getString("rol"))`, `rs.getTimestamp(...).toLocalDateTime()`) y arma un `Usuario`.
- Consultas: `findAll`, `findByRol`, `findById` y `findByCorreo`. Las de un solo resultado devuelven `Optional<Usuario>`.
- Verificaciones con `COUNT(*)`: `existsById`, `existsByCorreo`, `existsByTelefono`, y las variantes `...AndIdNot`. Estas últimas excluyen al propio usuario al actualizar, para que pueda conservar su correo.
- Escrituras: `insertar` (lee el id generado con `getGeneratedKeys()`), `actualizar`, `actualizarEstado` y `eliminar` (devuelven las filas afectadas por `executeUpdate()`).
- `contar(sql, params...)`: método privado que reutilizan todos los conteos; asigna los parámetros con `setObject`.
- Todos los métodos declaran `throws SQLException`.

Excepciones (`exception`)

`LanzadorExcepciones`: clase estática con un método `verificarX` por cada regla. Si la regla no se cumple, lanza su excepción propia; si se cumple, no hace nada.

Método	Regla	Excepción
`verificarCaracteres`	Solo letras (con tildes y ñ) y espacios, sin vacíos	`InvalidCharException`
`verificarCorreo`	Formato `usuario@dominio.ext`	`CorreoException`
`verificarTelefono`	Entre 7 y 15 dígitos	`TelefonoException`
`verificarTamanoContrasena`	Al menos 8 caracteres	`PasswordSizeException`
`verificarContrasena`	Una mayúscula, una minúscula y un número	`ContrasenaException`
`verificarRol`	Rol no nulo	`RolException`

Método	Regla	Excepción
`verificarRolPublico`	El registro público no admite `ADMINISTRADOR`	`RolNoPermitidoException`
`verificarEstado`	Estado no nulo	`EstadoException`
`verificarCorreoDuplicado`	El correo no debe existir	`CorreoDuplicadoException`
`verificarTelefonoDuplicado`	El teléfono no debe existir	`TelefonoDuplicadoException`
`verificarExistencia`	El registro debe existir	`RegistroNoEncontradoException`

Esas 11 excepciones extienden `Exception`, así que son *checked*: el compilador obliga a atraparlas o declararlas. Cada una fija su mensaje en español en el constructor. Hay una más, `AccesoDatosException`, que extiende `RuntimeException`. Envuelve la `SQLException` en las consultas de lectura, que no pueden devolver un código, y hace que Spring responda 500.

Servicios (`service`)

`CRUDOperation<T>`: la interfaz genérica heredada de TravelX. Define `create`, `updateById`, `deleteById`, `getAll`, `exist` y `count`, para que todos los servicios tengan el mismo contrato.

`UsuarioService`: implementa `CRUDOperation<UsuarioDTO>` y concentra la lógica de negocio. El patrón de cada escritura es: normalizar los datos, validar con `LanzadorExcepciones` dentro de un `try`, escribir con el repositorio y devolver 0. Cada `catch` devuelve un código distinto.

Código	Significado
0	Éxito
1	Nombre o apellido inválido
2	Correo inválido
3	Teléfono inválido
4	Contraseña corta
5	Contraseña sin mayúscula, minúscula o número
6	Rol no indicado
7	Rol no permitido en el registro público
8	Correo duplicado

Código	Significado
9	Teléfono duplicado
10	Usuario no encontrado
11	Error inesperado (por ejemplo, MySQL caído)

- `create`: lo usa el administrador; permite cualquier rol.
- `registrar`: lo usa el registro público; rechaza `ADMINISTRADOR` y fuerza el estado `ACTIVO`.
- `updateById`: si la contraseña llega vacía conserva la actual, y si el estado llega nulo conserva el actual.
- `updateEstado` (0 éxito, 1 estado nulo, 2 no encontrado, 3 error) y `deleteById` (0 éxito, 1 no encontrado, 2 tiene registros asociados, 3 error) tienen sus propios códigos.
- Consultas: `getAll`, `getAllByRol`, `getById` y `getByCorreo`. Las dos últimas devuelven `null` si no hay resultado. Si falla la BD lanzan `AccesoDatosException`. Toda conversión a DTO pasa por `toDto`, que borra el hash de la contraseña.
- `normalizar` quita espacios sobrantes y pasa el correo a minúsculas, para que `Ana@Mail.com` y `ana@mail.com` cuenten como el mismo correo.
- Si dos registros simultáneos pasan la verificación de duplicados, el `UNIQUE` de MySQL lanza `SQLIntegrityConstraintViolationException` con código 1062. `codigoDuplicado` la traduce a 8 o 9 según la restricción violada; con otro código devuelve 11.
- La contraseña se cifra con `PasswordEncoder` (BCrypt) antes de guardarla.

Utilidades (`util`)

`RespuestaUsuario`: contiene el `switch` que convierte los códigos 0–11 en `ResponseEntity` con mensaje en español y código HTTP: 400 datos inválidos, 403 rol no permitido, 409 duplicado, 404 no encontrado, 500 error. Está en un solo lugar para que crear, registrar, actualizar y "mi cuenta" respondan igual ante el mismo error.

Controladores (`controller`)

`AuthController` (`/auth`, público):

- `POST /auth/login`: autentica con `AuthenticationManager` y devuelve un `AuthResponseDTO` con el JWT. Responde 401 si las credenciales son malas, 403 si la cuenta está inactiva y 500 si falla la BD.
- `POST /auth/register`: llama a `usuarioService.registrar` y traduce el código con `RespuestaUsuario`; responde 201 si tiene éxito.

`UsuarioController` (`/usuario` , requiere token):

Endpoint	Rol	Qué hace
`POST /createjson`	ADMINISTRADOR	Crea un usuario con cualquier rol
`GET /getall` , `/getall/administradores` , `/getall/clientes` , `/getall/propietarios`	ADMINISTRADOR	Lista usuarios (202, o 204 si no hay)
`GET /getbyid/{id}` , `/getbycorreo?correo=`	ADMINISTRADOR	Busca un usuario
`PUT /updatejson?id=`	ADMINISTRADOR	Actualiza un usuario
`PUT /updateestado?id=&estado=`	ADMINISTRADOR	Activa o inactiva; no sobre la propia cuenta
`DELETE /deletebyid/{id}`	ADMINISTRADOR	Elimina; no la propia cuenta
`GET /count` , `/exists/{id}`	ADMINISTRADOR	Conteo y existencia
`GET /mi-cuenta` , `PUT /mi-cuenta`	Cualquier rol	Ver o editar los propios datos; el rol y el estado no se pueden cambiar

Los endpoints obtienen al usuario autenticado con `@AuthenticationPrincipal Usuario actual` , es decir, desde el token y no desde parámetros que el cliente pueda falsificar.

Seguridad (`security`)

`JwtUtil` : genera y valida tokens JWT firmados con HS256.

- El constructor lee `jwt.secret` y `jwt.expiration-ms` . Si la clave tiene menos de 32 caracteres, la aplicación no arranca.
- `generateToken` pone el correo como *subject* y agrega los *claims* `role` y `userId` .
- `extractUsername` , `extractRole` , `extractUserId` y `extractExpiration` leen el token verificando su firma.
- `validateToken` comprueba que el correo coincida y que el token no haya expirado.

`JwtAuthenticationFilter` : se ejecuta una vez por petición. Toma el encabezado `Authorization: Bearer <token>` , extrae el correo, carga el usuario desde la BD y, si la cuenta está activa y el token es válido, lo registra en el `SecurityContext` . Si el token es

inválido o el usuario ya no existe, no interrumpe la petición: la deja sin autenticar y `SecurityConfig` responde 401.

`UserDetailsServiceImpl`: puente entre Spring Security y el repositorio. `loadUserByUsername(correo)` llama a `findByCorreo` y lanza `UsernameNotFoundException` si no existe. Si la consulta falla, envuelve la `SQLException` en `InternalAuthenticationServiceException`, porque este método no puede lanzar excepciones *checked*.

`SecurityConfig`: define las reglas de acceso.

- Públicos: `OPTIONS`, `/error`, Swagger, `POST /auth/login` y `POST /auth/register`.
- `/usuario/mi-cuenta` para cualquier rol autenticado; el resto de `/usuario/**` solo para `ADMINISTRADOR`; cualquier otra ruta exige token.
- Sesiones `STATELESS`: el servidor no guarda sesión, todo viaja en el JWT. Por eso CSRF está desactivado.
- Respuestas propias: 401 "No autenticado" y 403 "No tiene permisos", en texto en español.
- CORS con los orígenes de `app.cors.allowed-origins`.
- Beans: `AuthenticationProvider` (usa `UserDetailsServiceImpl` + BCrypt), `AuthenticationManager` y `PasswordEncoder`.

Configuración (`configuration`)

`ConexionBD`: crea el `DataSource`, un pool de conexiones HikariCP, con la url, el usuario y la contraseña de `spring.datasource.*`. Es el único lugar donde se configura la conexión a MySQL (detalle en la sección de configuración JDBC).

`LoadDataBase`: un `CommandLineRunner` que al arrancar crea el administrador inicial con los datos de `app.admin.*`, si no existe. Si `ADMIN_PASSWORD` no está definida, no lo crea y lo avisa en el log; así no hay contraseñas en el código.

`OpenApiConfig`: configura Swagger UI (`/swagger-ui/index.html`) con la descripción de la API y el esquema `bearerAuth`, para probar endpoints protegidos con el botón *Authorize*.

Pruebas (`src/test`)

Clase	Qué prueba
`UsuarioServiceTest`	24 casos con Mockito: el repositorio y el codificador son simulados y el `ModelMapper` es real. Cubre cada código de error, la normalización del correo, que el registro público fuerce `ACTIVO`, que actualizar sin contraseña conserve el hash, que las consultas no devuelvan la contraseña y la traducción de `SQLIntegrityConstraintViolationException` (1062 → duplicado, 1451 → registros asociados, otro código → error)
`LanzadorExcepcionesTest`	11 casos: cada regla (las de Usuario y las nuevas de texto, rangos, montos, coordenadas, NIT, URI, fechas, dueño de foto y saldo) con un valor válido y uno inválido
`ReservaServiceTest`	13 casos: reserva correcta (total 315000 = tarifa 110000 + recargo 25000 + 2 servicios de 90000, hora de fin = inicio + 3 h), habitación inexistente, horario ocupado, tarifa de otro tipo, tarifa vencida, poca anticipación, servicio repetido o de otro motel, transiciones de estado inválidas, reembolso del 80 % (cliente) y del 100 % (propietario), cancelación sin pagos, cancelaciones no permitidas y la tarea automática con fecha límite en días hábiles
`AutorizacionServiceTest`	4 casos: el administrador pasa sin consultar la base, dueño y no dueño de un motel, recurso inexistente (deja pasar para que responda 404) y quién puede ver una reserva
`VesperaroomApplicationTests`	Levanta el contexto completo; necesita MySQL disponible

En total son **52 pruebas unitarias** y todas pasan con `mvnw test`.

Clases nuevas del modelo 3FN, paquete por paquete

Cada una de las 17 tablas nuevas sigue exactamente el mismo recorrido que Usuario: **entidad → DTO → repositorio JDBC → servicio con códigos int → Respuesta<X> → controlador**. Si se entiende Usuario, se entiende cualquier módulo; lo que cambia son los

campos y las reglas de negocio.

Entidades (entity) y DTOs (dto)

La entidad es el espejo de la fila en MySQL; el DTO es lo que viaja en el JSON. Se convierten con `ModelMapper` en modo `STRICT` para que un campo como `idMotel` no se confunda con `idMotelPropietario`. El dinero es `BigDecimal` (nunca `double`, que redondea mal) y las fechas son `LocalDate` / `LocalDateTime`.

Entidad	Tabla	Campos	Enum / notas
`Departamento`	departamento	idDepartamento, nombre	Catálogo
`Ciudad`	ciudad	idCiudad, idDepartamento, nombre	FK a departamento
`Caracteristica`	caracteristica	idCaracteristica, nombre, icono	Catálogo (wifi, jacuzzi…)
`Modalidad`	modalidad	idModalidad, nombre, duracionHoras	Catálogo (rato, amanecida…)
`PerfilPropietario`	perfil_propietario	idUsuario (PK y FK), nit, razonSocial, banco, tipoCuenta, numeroCuenta, fechaVerificacion	`TipoCuenta`; relación 1:1 con usuario
`Motel`	motel	idMotel, idPropietario, idCiudad, nombre, descripcion, calle, carrera, latitud, longitud, estado	`Estado`
`PoliticaReserva`	politica_reserva	idMotel (PK y FK), diasHabilesDevolucion, porcentajeReembolso, capacidadMaxSinRecargo, tarifaPersonaAdicional, anticipacionMinimaHoras, tiempoGraciaNoShowMin	1:1 con motel
`TipoHabitacion`	tipo_habitacion	idTipo, idMotel, nombre, capacidadPersonas, descripcion	Características por tabla intermedia
`Habitacion`	habitacion	idHabitacion, idMotel, idTipo, numero, piso, estado	`Estado`; FK compuesta (idTipo, idMotel)
`Foto`	foto	idFoto, idMotel, idHabitacion, uri, orden	Solo uno de los dos dueños (CHECK)

Entidad	Tabla	Campos	Enum / notas
`Tarifa`	tarifa	idTarifa, idTipo, idModalidad, precio, tipoDia, vigenteDesde, vigenteHasta	`TipoDia`
`ServicioAdicional`	servicio_adicional	idServicio, idMotel, nombre, descripcion, precio, activo	
`Reserva`	reserva	idReserva, codigoReserva, idUsuario, idHabitacion, idTarifa, cantidadPersonas, fechaCreacion, fechaHoraInicio, fechaHoraFin, estado, total	`Estado`
`ReservaServicio`	reserva_servicio	idReserva, idServicio, cantidad, precioUnitario	Detalle de la reserva; guarda el precio del momento
`Pago`	pago	idPago, idReserva, monto, fechaPago, metodoPago, estado, referenciaTransaccion	`MetodoPago`, `Estado`
`Resena`	resena	idResena, idReserva, calificacion, comentario, fecha	Una por reserva (UNIQUE)
`RespuestaPropietario`	respuesta_propietario	idResena (PK y FK), comentario, fecha	1:1 con reseña

Los DTOs tienen los mismos campos más las validaciones de Jakarta (`@NotNull` , `@Size` …) y, en `ReservaDTO` , la lista `servicios` de `ReservaServicioDTO` (idServicio, cantidad). Los campos que calcula el servidor (total, código, hora de fin, fechas de creación) se ignoran si el cliente los envía.

Repositorios (repository)

Todos usan el mismo esqueleto de `UsuarioRepository` : `try-with-resources` sobre `Connection` , `PreparedStatement` y `ResultSet` , un método privado `mapear(ResultSet)` que arma la entidad y SQL escrito a mano. Métodos comunes: `findAll` , `findById` , `insertar` , `actualizar` , `eliminar` , `existsById` , `count` , y los `existsBy…` que revisan duplicados antes de insertar (por ejemplo `existsByMotelAndNombre`). Además:

Repositorio	Métodos propios
`CiudadRepository`	`findByDepartamento`
`MotelRepository`	`findActivos` , `findActivosByCiudad` , `findByPropietario` , `actualizarEstado`

Repositorio	Métodos propios
`TipoHabitacionRepository`	`findByMotel`, `findCaracteristicas`, `asignarCaracteristica`, `quitarCaracteristica` (tabla intermedia)
`HabitacionRepository`	`findByMotel`, `findByTipo`, `findDisponibles` (usa `NOT EXISTS` contra reservas que se cruzan), `actualizarEstado`
`FotoRepository`	`findByMotel`, `findByHabitacion`
`TarifaRepository`	`findByTipo`
`ServicioAdicionalRepository`	`findByMotel`, `findActivosByMotel`
`ReservaRepository`	`findByUsuario`, `findByMotel`, `findServicios`, `existsSolapamiento`, `insertarConServicios` (transacción con `SELECT … FOR UPDATE` y `addBatch`), `actualizarEstado`
`PagoRepository`	`findByReserva`, `sumarMontoActivo` (con `COALESCE`), `actualizarEstado`
`ResenaRepository`	`findByMotel`, `findByUsuario`, `promedioByMotel` (con `AVG`)
`AutorizacionRepository`	`propietarioDeMotel`, `…DeTipo`, `…DeHabitacion`, `…DeTarifa`, `…DeServicio`, `…DeFoto`, `…DeReserva`, `…DePago`, `…DeResena` y `clienteDeReserva`, `…DePago`, `…DeResena`: hacen `JOIN` hasta `motel.id_propietario` o `reserva.id_usuario` y devuelven `Optional<Long>`

Las columnas opcionales se leen con `rs.getObject(col, Long.class)` (devuelve `null` en vez de 0) y se escriben con `ps.setObject(i, valor, Types.BIGINT)`.

Servicios (service)

Los servicios de catálogo y de propietario implementan `CRUDOperation<DTO>` igual que `UsuarioService`: `create`, `getAll`, `getById`, `updateById`, `deleteById`, `count`, `exist`. Cada método valida con `LanzadorExcepciones`, atrapa las excepciones y devuelve un **código int** documentado en su Javadoc (0 = éxito). Los errores de MySQL se traducen con `CodigoMySQL`: 1062 → duplicado, 1451 → tiene registros asociados, 1452 → referencia inexistente.

Servicio	Métodos propios y reglas importantes
`DepartamentoService`, `ModalidadService`, `CaracteristicaService`	Solo CRUD; nombre único
`CiudadService`	`getAllByDepartamento`; verifica que el departamento exista
`PerfilPropietarioService`	`verificar` (marca la fecha de verificación); valida NIT, número de cuenta y que el usuario tenga rol PROPIETARIO
`MotelService`	`getActivos`, `getActivosByCiudad`, `getByPropietario`, `updateEstado`; valida coordenadas
`PoliticaReservaService`	Rangos de porcentajes, horas y minutos
`TipoHabitacionService`	`getByMotel`, `getCaracteristicas`, `asignarCaracteristica`, `quitarCaracteristica`
`HabitacionService`	`getByMotel`, `getByTipo`, `getDisponibles`, `updateEstado`; el tipo debe ser del mismo motel
`FotoService`	`getByMotel`, `getByHabitacion`; exactamente un dueño y URI http/https
`TarifaService`	`getByTipo`; vigencia coherente (desde ⩽ hasta) y precio positivo
`ServicioAdicionalService`	`getByMotel`, `getActivosByMotel`
`ReservaService`	`crear` (códigos 0–12), `cambiarEstado`, `getByUsuario`, `getByMotel`, `deleteById`. No implementa CRUDOperation porque una reserva no se edita: solo cambia de estado
`PagoService`	`crear` (códigos 0–8: no pagar más del saldo, solo reservas vigentes), `cambiarEstado`, `getByReserva`
`ResenaService`	`crear` (solo el cliente de una reserva FINALIZADA, una por reserva), `getByMotel`, `getByUsuario`, `getPromedioByMotel`

Servicio	Métodos propios y reglas importantes
`RespuestaPropietarioService`	Solo el dueño del motel reseñado responde
`AutorizacionService`	`esAdministrador`, `puedeGestionar{Motel, Tipo, Habitacion, Tarifa, Servicio, Foto, Reserva, Pago}`, `puedeVerReserva`, `esClienteDeReserva`, `puedeVerPago`, `puedeResponderResena`, `puedeEditarResena`

Excepciones (exception)

Se sumaron 23 excepciones checked, todas lanzadas desde métodos `verificarX` estáticos de `LanzadorExcepciones`, igual que en TravelX. Algunas son genéricas y llevan un dato que el servicio lee para elegir el código: `TextoInvalidoException.getCampo()`, `ReferenciaNoEncontradaException.getEntidad()`, `RegistroDuplicadoException.getDetalle()`.

Validación	Excepción
`verificarTexto`, `verificarTextoOpcional`	`TextoInvalidoException`
`verificarRango`, `verificarMonto`	`ValorFueraDeRangoException`
`verificarObligatorio`	`CampoObligatorioException`
`verificarReferencia`	`ReferenciaNoEncontradaException`
`verificarDuplicado`	`RegistroDuplicadoException`
`verificarCoordenadas`	`CoordenadaException`
`verificarUri`	`UriException`
`verificarNit`	`NitException`
`verificarNumeroCuenta`	`CuentaBancariaException`
`verificarVigencia`, `verificarFechaFutura`	`FechaException`
`verificarDuenoFoto`	`FotoDuenoException`
`verificarRolPropietario`	`RolPropietarioException`
`verificarHabitacionDisponible`	`HabitacionNoDisponibleException`
`verificarHorarioLibre`	`HorarioOcupadoException`

Validación	Excepción
`verificarTarifaAplicable`	`TarifaNoAplicableException`
`verificarTarifaVigente`	`TarifaNoVigenteException`
`verificarAnticipacion`	`AnticipacionException`
`verificarCapacidad`	`CapacidadExcedidaException`
`verificarServicio`	`ServicioNoValidoException`
`verificarTransicion`	`TransicionEstadoException`
`verificarEstadoReserva`	`EstadoReservaException`
`verificarSaldo`	`MontoExcedidoException`
`verificarPropiedadReserva`	`PropiedadReservaException`

Utilidades (util)

Hay una clase `Respuesta<Entidad>` por módulo (16 nuevas) con el mismo `switch` de `RespuestaUsuario`: recibe el código int del servicio y devuelve el `ResponseEntity` con el estado HTTP y el mensaje en español. Lo repetido entre todas (listas vacías, no encontrado, eliminar, cambiar estado, conteo, existe, 403) está en `RespuestaGeneral`. `Normalizador` limpia espacios y pasa correos a minúscula; `CodigoMySQL` guarda las constantes 1062, 1451 y 1452.

Controladores (controller)

Todos usan las rutas de TravelX: `/createjson` (POST), `/updatejson` (PUT), `/getall`, `/getbyid/{id}`, `/deletebyid/{id}`, `/count`, `/exists/{id}`. Antes de llamar al servicio, los controladores de recursos de propietario preguntan a `AutorizacionService` si el usuario del token es el dueño; si no, responden 403.

Controlador	Ruta base	Rutas propias
`DepartamentoController`	/departamento	—
`CiudadController`	/ciudad	/getbydepartamento/{idDepartamento}
`CaracteristicaController`	/caracteristica	—
`ModalidadController`	/modalidad	—
`PerfilPropietarioController`	/perfilpropietario	/mi-perfil, /verificar
`MotelController`	/motel	/getall/todos, /getbyciudad/{idCiudad}, /mis-moteles, /updateestado

Controlador	Ruta base	Rutas propias
`PoliticaReservaController`	/politica	—
`TipoHabitacionController`	/tipohabitacion	/getbymotel/{idMotel}, /caracteristicas/{id}, /asignarcaracteristica, /quitarcaracteristica
`HabitacionController`	/habitacion	/getbymotel/{idMotel}, /getbytipo/{idTipo}, /disponibles, /updateestado
`FotoController`	/foto	/getbymotel/{idMotel}, /getbyhabitacion/{idHabitacion}
`TarifaController`	/tarifa	/getbytipo/{idTipo}
`ServicioAdicionalController`	/servicio	/getbymotel/{idMotel}, /getbymotel/{idMotel}/todos
`ReservaController`	/reserva	/mis-reservas, /getbymotel/{idMotel}, /updateestado, /cancelar/{id}
`PagoController`	/pago	/getbyreserva/{idReserva}, /updateestado
`ResenaController`	/resena	/getbymotel/{idMotel}, /promedio/{idMotel}, /mis-resenas
`RespuestaPropietarioController`	/respuesta	—

Seguridad (security)

`SecurityConfig` define tres grupos de rutas, evaluados en orden: `CATALOGO_PUBLICO` (consultas GET sin token, para que un visitante pueda buscar moteles), `SOLO_ADMIN` (listados globales y borrado de reservas) y el resto por rol: catálogos solo ADMINISTRADOR; moteles, habitaciones, tarifas, fotos y servicios PROPIETARIO o ADMINISTRADOR; reservas, pagos y reseñas cualquier usuario autenticado. Que el propietario toque solo lo suyo no lo decide SecurityConfig sino `AutorizacionService`.

Scripts SQL y ejecución

El esquema se crea con los scripts de la carpeta `database/`, en este orden. El backend nunca crea ni modifica tablas.

Orden	Script	Contenido
1	`01_creacion_objetos.sql`	Crea la base `motelnow` y las 19 tablas con sus PK, FK (incluida la compuesta de `habitacion`), `UNIQUE`, `CHECK` e índices. Hace `DROP TABLE IF EXISTS`, así que borra los datos existentes

Orden	Script	Contenido
2	`02_insercion_registros.sql`	Datos de prueba de todas las tablas con ids explícitos: 6 usuarios (contraseña `Vespera2026`), 3 moteles, 9 habitaciones, 10 tarifas, 4 reservas con pagos y reseñas
3	`03_scripts_varios.sql`	Usuario MySQL de la aplicación (comentado), vista `v_reserva_detalle` y consultas: disponibilidad con `NOT EXISTS`, ingresos por motel, promedio de calificaciones, saldo pendiente

Para una base que ya tiene datos están las migraciones de `database/migraciones/`, que se ejecutan a mano con `root`:

Script	Qué hace
`V1__usuario.sql`	Estado inicial: crea `usuario` solo si no existe
`V2__esquema_3fn.sql`	Agrega las 18 tablas nuevas con `CREATE TABLE IF NOT EXISTS` y carga los catálogos base con `INSERT IGNORE`. No toca los usuarios existentes y puede ejecutarse dos veces
`R2__revertir_esquema_3fn.sql`	Revierte V2: borra las 18 tablas en orden inverso a sus dependencias

Los nombres siguen la convención de Flyway (`V<versión>__<descripción>.sql`), pero el proyecto no usa Flyway: es solo una forma ordenada de versionar el esquema.

Variables de entorno que lee `application.properties`:

Variable	Obligatoria	Por defecto	Uso
`DB_PASSWORD`	Sí	—	Contraseña del usuario MySQL
`JWT_SECRET`	Sí	—	Clave de firma del JWT; mínimo 32 caracteres o la aplicación no arranca

Variable	Obligatoria	Por defecto	Uso
`DB_HOST` / `DB_PORT` / `DB_USER`	No	`localhost` / `3306` / `usuario`	Ubicación y usuario de MySQL (la IP de la máquina virtual, cuando esté lista)
`CORS_ORIGINS`	No	`http://localhost:4200`	Orígenes del frontend separados por coma
`ADMIN_PASSWORD`	No	vacío	Si se define, se crea el administrador inicial al arrancar
`ADMIN_CORREO` / `ADMIN_TELEFONO`	No	`admin@vesperaroom.com` / `3000000000`	Datos de ese administrador

Con MySQL arriba y los scripts ejecutados, `./mvnw spring-boot:run` levanta la API en el puerto 8080 y Swagger queda en `/swagger-ui/index.html`.